

Concurrencia y Paralelismo

Clase 1



Facultad de Informática
UNLP

Metodología del curso

♦ Horarios:

- Teorías: viernes de 11 a 14 (aula 9)
- Explicación Práctica: jueves de 11 a 13 hs. - aula 14
- Prácticas: jueves de 17.30 a 20 hs. - aula 14

♦ Cursada:

- La cursada se aprueba con un parcial práctico con dos temas independientes que cuenta con dos recuperatorios. La primera fecha se encuentra distribuida.

♦ Promoción:

- Aprobar la cursada
- La suma de las notas de cada tema aprobado del parcial (máximo 4 puntos cada uno) más la nota de un examen teórico opcional (máximo 2 puntos) debe ser ≥ 6 .

♦ Final: examen teórico / práctico.

Metodología del curso

- ◆ **Comunicación:** Plataforma IDEAS (ideas.info.unlp.edu.ar).
- ◆ **Bibliografía / material:**
 - ***Libro base:*** Foundations of Multithreaded, Parallel, and Distributed Programming. Gregory Andrews. Addison Wesley. (www.cs.arizona.edu/people/greg/mpdbook).
 - An Introduction to Parallel Computing. Design and Analysis of Algorithms, 2/E. Grama, Gupta, Karypis, Kumar. Pearson Addison Wesley.
- ◆ **Planteo de temas/ejercicios:** recomendado hacerlos.

Temas del curso

- ◆ **Conceptos básicos.** Concurrencia y arquitecturas de procesamiento. Multithreading, Procesamiento Distribuido, Procesamiento Paralelo.
- ◆ **Concurrencia por memoria compartida.** Procesos y sincronización. Locks y Barreras. Semáforos. Monitores. Resolución de problemas concurrentes con sincronización por MC.
- ◆ **Concurrencia por pasaje de mensajes (MP).** Mensajes asincrónicos. Mensajes sincrónicos. Remote Procedure Call (RPC). Rendezvous. Paradigmas de interacción entre procesos.
- ◆ **Lenguajes que soportan concurrencia.** Características. Similitudes y diferencias.
- ◆ **Introducción a la programación paralela.** Conceptos, herramientas de desarrollo, aplicaciones.



Introducción

Concurrencia

¿Que es?

- ◆ Concurrencia es la capacidad de ejecutar múltiples actividades en paralelo o simultáneamente.
- ◆ Permite a distintos objetos actuar al mismo tiempo.
- ◆ Factor relevante para el diseño de hardware, sistemas operativos, multiprocesadores, computación distribuida, programación y diseño.

¿Donde está?

- ◆ Navegador Web accediendo una página mientras atiende al usuario.
- ◆ Varios navegadores accediendo a la misma página.
- ◆ Acceso a disco mientras otras aplicaciones siguen funcionando.
- ◆ Impresión de un documento mientras se consulta.
- ◆ El teléfono avisa recepción de llamada mientras se habla.
- ◆ Varios usuarios conectados al mismo sistema (reserva de pasajes).
- ◆ Cualquier objeto más o menos “inteligente” exhibe concurrencia.
- ◆ Juegos, automóviles, etc.

¿Por qué es necesaria la concurrencia?

- Aplicaciones con estructura más natural (si los problemas son concurrentes, es más natural resolverlos de forma concurrente).
- No hay más ciclos de reloj → Multicore → ¿cómo aprovecharlo?
- Mejora en la respuesta – mejora el rendimiento
- Sistemas distribuidos

Concurrencia “natural”

En algunos casos se tiende a pensar en sistemas secuenciales en lugar de concurrentes para simplificar el proceso de diseño. Pero esto va en contra de la necesidad de sistemas de cómputo cada vez más poderosos y flexibles.

- ♦ **Problema 1:** Desplegar cada 3 segundos un cartel ROJO.
- ♦ **Solución secuencial:**

Programa Cartel

Mientras (true)

Demorar (3 seg)

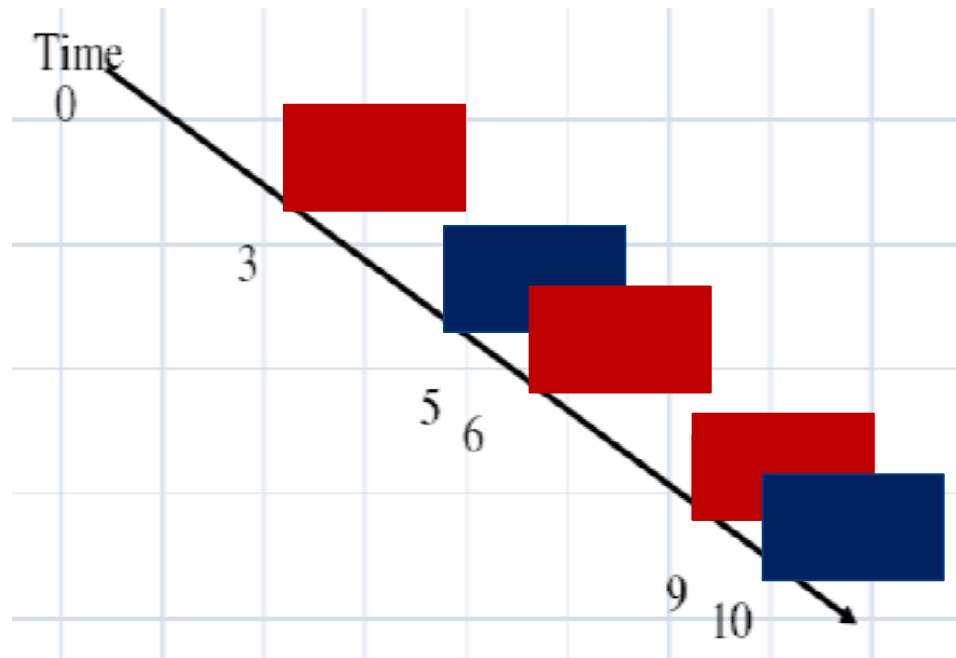
Desplegar cartel

Fin mientras

Fin programa

Concurrencia “natural”

- ◆ **Problema 2:** Desplegar cada 3 segundos un cartel ROJO y cada 5 segundos un cartel AZUL.



Concurrencia “natural”

Programa Carteles

Proximo_Rojo = 3

Proximo_Azul = 5

Actual = 0

Mientras (true)

Si (Proximo_Rojo < Proximo_Azul)

Demorar (Proximo_Rojo – Actual)

Desplegar cartel ROJO

Actual = Proximo_Rojo

Proximo_Rojo = Proximo_Rojo +3

sino

Demorar (Proximo_Azul – Actual)

Desplegar cartel AZUL

Actual = Proximo_Azul

Proximo_Azul = Proximo_Azul +5

Fin mientras

Fin programa

Concurrencia “natural”

- ◆ Obliga a establecer un orden en el despliegue de cada cartel.
- ◆ Código más complejo de desarrollar y mantener.
- ◆ ¿Qué pasa si se tienen más de dos carteles?
- ◆ **Más natural:** cada cartel es un elemento independiente que actúa concurrentemente con otros → *es decir, ejecutar dos o más procesos concurrentemente.*

Process Cartel (color, tiempo)

Mientras (true)

Demorar (**tiempo** segundos)

Desplegar cartel (color)

Fin mientras

Fin programa

- ◆ No hay un orden preestablecido en la ejecución ⇒ **no determinismo** (ejecuciones con la misma “entrada” puede generar diferentes “salidas”)

Procesamiento secuencial, concurrente y paralelo

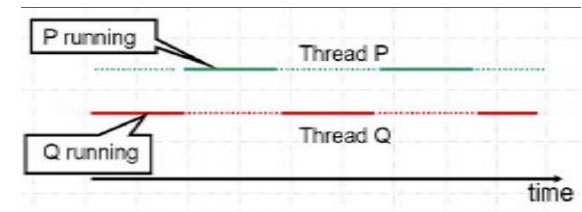
Supongamos que tenemos una unidad de procesamiento (core) para realizar N tareas.

Puede trabajar de acuerdo a dos enfoques diferentes:

- ♦ **Solución secuencial:** hace las tareas de a una a la vez, hasta no terminar una no se puede comenzar el siguiente. La solución secuencial **nos fuerza** a establecer un **estricto orden temporal**.
- ♦ **Solución concurrente (y no paralela):** dedica una parte del tiempo a cada tarea, aprovechando tiempos ociosos en la realización de cada uno de ellas
⇒ **Concurrencia sin paralelismo de hardware.**

Dificultades ⇒

- Distribución de carga de trabajo.
- Necesidad de compartir recursos.
- Necesidad de esperarse en puntos clave.
- Necesidad de comunicarse.
- Necesidad de recuperar el “estado” de cada proceso al retomarlo.



Procesamiento secuencial, concurrente y paralelo

Supongamos que tenemos N unidades de procesamiento (cores) para hacer las mismas N tareas.

- ♦ **Solución paralela (además es concurrente):** todos los cores trabajan al mismo tiempo haciendo cada uno una única tarea $\Rightarrow$ **Concurrencia con paralelismo de hardware**. Sigue siendo concurrente, pero además es paralelo.

Consecuencias $\Rightarrow$

- Menor tiempo para completar el trabajo.
- Menor esfuerzo individual.
- Paralelismo del hardware.

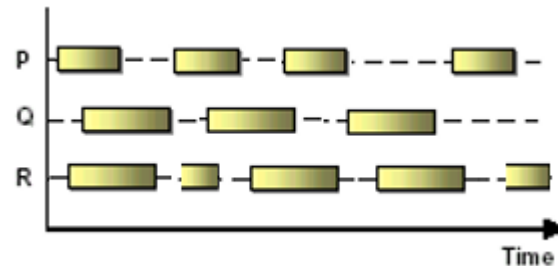
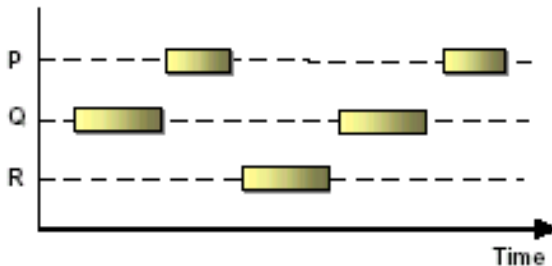
Dificultades $\Rightarrow$

- Distribución de la carga de trabajo (en relación con la arquitectura y el trabajo)
- Necesidad de compartir recursos evitando conflictos.
- Necesidad de esperarse en puntos clave.
- Necesidad de comunicarse.
- Tratamiento de las fallas.

CONCURRENCIA $\Rightarrow$ Concepto de software no restringido a una arquitectura particular de hardware ni a un número determinado de procesadores.

Programa Concurrente

- Por ahora llamaremos “*Proceso*” a elemento computacional que ejecuta un único flujo de control en forma secuencial: ejecuta una instrucción y cuando esta finaliza ejecuta la siguiente.
- **Programa Concurrente:** múltiples procesos que trabajan “simultáneamente” para resolver un mismo problema y/o utilizar recursos compartidos.



Programa Concurrente

- Un programa concurrente puede tener N *procesos* habilitados para ejecutarse concurrentemente y un *sistema concurrente* puede disponer de M *elementos de procesamiento* ($M \geq 1$) cada uno de los cuales puede ejecutar uno o más procesos.
- Cuando hay más de un elemento de procesamiento ($M > 1$) además de *Sistema Concurrente* es un *Sistema Paralelo*.
- Cuando hay sólo UN elemento de procesamiento ($M = 1$) es un *Sistema Concurrente* pero NO un *Sistema Paralelo*.
 - El tiempo de CPU es compartido entre varios procesos, por ejemplo, por *time slicing*.
 - El sistema operativo controla y planifica procesos: si el slice expiró o el proceso se bloquea el sistema operativo hace *context (process) switch*.

Posibles comportamientos de los procesos

Múltiples procesos trabajando “simultáneamente” pueden actuar de forma independiente, cooperar y/o competir entre ellos.

Procesos independientes

- Relativamente raros.
- Poco interesante.

Competencia

- Típico en Sistemas Operativos y Redes.
- Compiten por el uso de recursos compartidos.

Cooperación

- Los procesos colaboran para resolver una tarea común.
- Interacción entre los procesos (Sincronización y comunicación).

Diferencia entre procesos e hilos

- **Procesos:** Cada proceso tiene su propio espacio de direcciones y recursos.
- **Procesos livianos, threads o hilos:**
 - Proceso “liviano” que tiene su propio contador de programa y su pila de ejecución, pero no controla el “contexto pesado” (por ejemplo, las tablas de página).
 - Todos los hilos de un proceso comparten el mismo espacio de direcciones y recursos (los del proceso).
 - El programador o el lenguaje deben proporcionar mecanismos para evitar interferencias.
 - La concurrencia puede estar provista por el lenguaje (Java) o por el Sistema Operativo (C/POSIX).



Conceptos básicos de concurrencia

Conceptos básicos de concurrencia

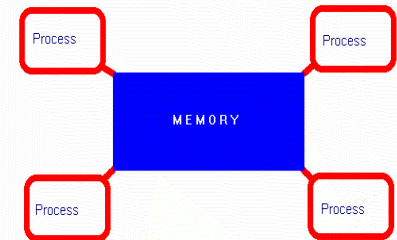
Comunicación entre procesos

La comunicación entre procesos concurrentes indica el modo en que se organizan y transmiten datos entre tareas concurrentes. Esta organización requiere especificar **protocolos** para controlar el progreso y la corrección. Los procesos se **COMUNICAN**:

- Por *Memoria Compartida*.
- Por *Pasaje de Mensajes*.

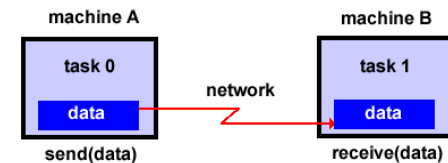
• Memoria compartida

Los procesos intercambian información sobre la memoria compartida o actúan coordinadamente sobre datos residentes en ella. Se debe asegurar de no operar simultáneamente sobre la memoria compartida.



• Pasaje de mensajes

Es necesario establecer un **canal** (lógico o físico) para transmitir información entre procesos. Se debe tener un protocolo adecuado. Para que la comunicación sea efectiva los procesos deben “saber” cuándo tienen mensajes para leer y cuando deben transmitir mensajes.



Conceptos básicos de concurrencia

Sincronización entre procesos

La **sincronización** es la posesión de información acerca de otro proceso para coordinar actividades. Los procesos se sincronizan:

- Por *exclusión mutua*.
- Por *condición*.

- **Sincronización por exclusión mutua**

- Asegurar que sólo un proceso tenga acceso a un recurso compartido en un instante de tiempo.
- Si el programa tiene **secciones críticas** que pueden compartir más de un proceso, la exclusión mutua evita que dos o más procesos puedan encontrarse en la misma sección crítica al mismo tiempo.

- **Sincronización por condición**

- Permite bloquear la ejecución de un proceso hasta que se cumpla una condición dada.

Conceptos básicos de concurrencia

Interferencia

Interferencia: un proceso toma una acción que invalida las suposiciones hechas por otro proceso.

Ejemplo 1: nunca se debería dividir por 0.

```
int x, y, z;
```

```
process A1
```

```
{ ....  
  y = 0;  
  ....  
}
```

```
process A2
```

```
{ .....  
  if (y <> 0) z = x/y;  
  .....  
}
```

Ejemplo 2: siempre *Público* debería terminar con valor igual a $E1 + E2$.

```
int Público = 0
```

```
process B1
```

```
{ int E1 = 0;  
  for i= 1..100  
  { esperar llegada  
    E1 = E1 + 1;  
    Público = Público + 1;  
  }  
}
```

```
process B2
```

```
{ int E2 = 0;  
  for i= 1..100  
  { esperar llegada  
    E2 = E2 + 1;  
    Público = Público + 1;  
  }  
}
```

Conceptos básicos de concurrencia

Manejo de los recursos

Uno de los temas principales de la programación concurrente es la **administración de recursos compartidos**:

- Esto incluye la asignación de recursos compartidos, métodos de acceso a los recursos, bloqueo y liberación de recursos, seguridad y consistencia.
- Una propiedad deseable en sistemas concurrentes es el equilibrio en el acceso a recursos compartidos por todos los procesos (***fairness***).
- Dos situaciones NO deseadas en los programas concurrentes son la ***inanición*** de un proceso (no logra acceder a los recursos compartidos) y el ***overloading*** de un proceso (la carga asignada excede su capacidad de procesamiento).
- Otro problema importante que se debe evitar es el ***deadlock***.

Conceptos básicos de concurrencia

Problema de deadlock



Dos (o más) procesos pueden entrar en *deadlock*, si por error de programación ambos se quedan esperando que el otro libere un recurso compartido. La ausencia de deadlock es una propiedad necesaria en los procesos concurrentes.

Conceptos básicos de concurrencia

Requerimientos para un lenguaje concurrente

Independientemente del mecanismo de comunicación / sincronización entre procesos, los **lenguajes de programación concurrente** deberán proveer primitivas adecuadas para la especificación e implementación de las mismas.

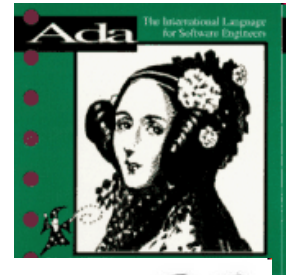
- **Requerimientos de un lenguaje de programación concurrente:**

- Indicar las tareas o procesos que pueden ejecutarse concurrentemente.
- Mecanismos de sincronización.
- Mecanismos de comunicación entre los procesos.



OCCAM

Modula-2



GO



OpenMP



Problemas asociados con la Programación Concurrente

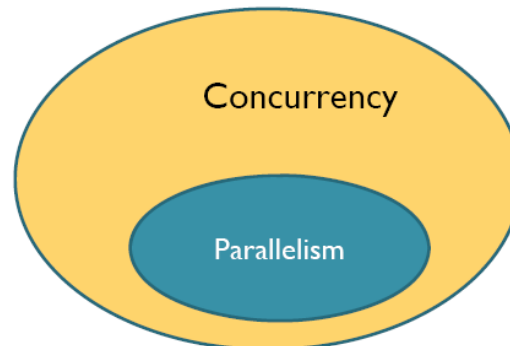
- ♦ Los procesos no son independientes y comparten recursos. La necesidad de utilizar mecanismos de exclusión mutua y/o sincronización agrega complejidad a los programas.
- ♦ Hay un **no determinismo** implícito en el interleaving de procesos concurrentes. Esto significa que dos ejecuciones del mismo programa no necesariamente son idénticas $\Rightarrow$ *dificultad para la interpretación y debug*.
- ♦ Posible reducción de performance por **overhead** de context switch, comunicación, sincronización, ...
- ♦ Mayor tiempo de desarrollo y puesta a punto. Difícil paralelizar algoritmos secuenciales.
- ♦ Necesidad de adaptar el software concurrente al hardware paralelo para mejora real en el rendimiento.

Conceptos básicos de concurrencia

Concurrencia y Paralelismo

CONCURRENCIA $\Rightarrow$ Concepto de software no restringido a una arquitectura particular de hardware ni a un número determinado de procesadores. Especificar la concurrencia implica especificar los *procesos concurrentes*, su *comunicación* y su *sincronización*.

PARALELISMO $\Rightarrow$ Se asocia con la ejecución concurrente en múltiples procesadores con el objetivo principal de reducir el tiempo de ejecución.





Clases de Instrucciones

Clases de instrucciones

Un programa concurrente está formado por un conjunto de “procesos” secuenciales.

- La programación secuencial estructurada puede expresarse con 3 clases de instrucciones básicas: **asignación**, **alternativa** (decisión) e **iteración** (repetición con condición).
- Se requiere una clase de instrucción para representar la concurrencia.

Además de las habituales estructuras de control y asignación, utilizaremos algunas instrucciones y estructuras de control particulares para trabajar con programación concurrente.

Clases de instrucciones

skip: termina inmediatamente y no tiene efecto sobre ninguna variable de programa.

Sentencias de alternativa múltiple:

if $B1 \rightarrow S1$

$\square B2 \rightarrow S2$

....

$\square Bn \rightarrow Sn$

fi

Elección no determinística.

Si ninguna guarda es verdadera el *if* no tiene efecto.

Sentencias de alternativa ITERATIVA múltiple:

do $B1 \rightarrow S1$

$\square B2 \rightarrow S2$

....

$\square Bn \rightarrow Sn$

od

Las sentencias guardadas son evaluadas y ejecutadas hasta que todas las guardas sean falsas.

La elección es no determinística si más de una guarda es verdadera.

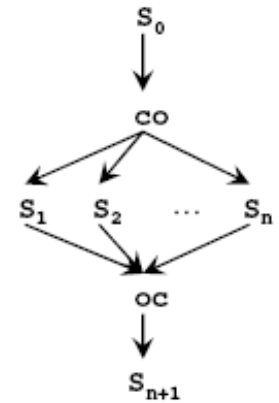
Clases de instrucciones

CONCURRENCIA

- Sentencia **co**:
co S1 // // Sn oc → Ejecuta las S_i tareas concurrentemente.
La ejecución del **co** termina cuando todas las tareas terminaron.

Cuantificadores.

co [i=1 to n] { a[i]=0; b[i]=0 } oc → Crea n tareas concurrentes.



- **Process**: otra forma de representar concurrencia
process A {sentencias} → proceso único independiente.
Cuantificadores.
process B [i=1 to n] {sentencias} → n procesos independientes.
- **Diferencia**: **process** ejecuta en **background**, mientras el código que contiene un **co** espera a que el proceso creado por la sentencia **co** termine antes de ejecutar la siguiente sentencia.

Clases de instrucciones

Programación secuencial y concurrente

Ejemplo: ¿qué imprime en cada caso? ¿son equivalentes?

```
process imprime10
{
    for [i=1 to 10] write(i);
}
```

```
process imprime1 [i= 1..10]
{
    write(i);
}
```

No determinismo....